

AI Rules Explained

FRONTMATTER

title: AI Rules Explained
video no: 17 of 30
series: Building an AI Business From Scratch
voice_reference: ../../Input/4. Resources/2.
scripting_resources/voice_reference/nate_herk_voice_reference.md
video_type: concept
hook_type: Problem-Statement
script_versions: A + B
status: DRAFT

REFERENCE VIDEOS

- CLAUDE CODE FULL COURSE 4 HOURS: Build & Sell (2026) (<https://www.youtube.com/watch?v=QoQBzR1Nlql>) -- Nick Saraev -- Watch the CLAUDE.md section for how rules are written and how Claude actually applies them in practice
- "I built 500+ AI agents, here's everything I know" (<https://www.youtube.com/watch?v=Svp7fbF0g2l>) -- Nate Herk -- Watch for how Nate structures agent constraints: what he puts in rules vs what he leaves to the agent's judgment
- Claude Code Hooks: Complete Guide (<https://claudefa.st/blog/tools/hooks/hooks-guide>) -- ClaudeFast -- Read alongside this video: rules and hooks work together, and this guide shows where rules end and hooks begin

THE ONE THING

AI rules files are the guardrails that prevent your agent from making decisions you did not authorise -- they are how you stay in control without watching every step.

HOOK

Your AI agent will do exactly what you told it to do. The problem is you probably did not tell it what NOT to do. Rules files fix that -- and they are simpler than you think.

TARGET VIEWER

Someone building AI agents or Claude Code workflows who keeps running into situations where Claude does something technically correct but not what they wanted -- the problem is missing constraints, not missing intelligence.

PAYOFF

After watching, the viewer knows what an AI rules file is, the three categories every rules file needs, and can write one for their current agent that immediately reduces unwanted autonomous decisions.

BEAT STRUCTURE

Beat -- The Permission Problem

Point: Agents do what they are allowed to do. Without explicit rules, "allowed" defaults to "everything in scope" -- which is always too broad for production use.

Visual: Agent running a task, making a plausible but unwanted decision -- technically within scope, but not what was intended. No rules = agent decides.

Target words: ~100w

Beat -- What a Rules File Is

Point: A rules file is a set of explicit constraints given to an agent: what it can do, what it cannot do, how to handle edge cases, and when to ask instead of act.

Visual: Simple rules file open in editor: three sections -- Always, Never, Escalate -- each with 3-5 specific examples

Target words: ~150w

Beat -- Rules in Claude Code

Point: In Claude Code, rules live in CLAUDE.md -- but you can scope them to subfolders with local CLAUDE.md files. Claude reads the most local rules first, then walks up to the root.

Visual: File tree: root CLAUDE.md (global rules) + /agents/ subfolder with its own CLAUDE.md (agent-specific rules) -- show both open simultaneously

Target words: ~160w

Beat -- The Three Categories

Point: Always (must do this), Never (must not do this), Escalate (stop and ask me before doing this). These three cover 90 percent of what any rules file needs.

Visual: Three-column table: Always / Never / Escalate -- with specific real examples in each column. Keep it scannable.

Target words: ~190w

Beat -- Rules vs Permissions -- The Difference

Point: Permissions control which tools Claude can access. Rules control how Claude uses them. Both matter. Permissions are technical guardrails; rules are behavioural ones.

Visual: Side-by-side: settings.local.json (technical permissions) vs CLAUDE.md rules section (behavioural constraints) -- show they complement each other

Target words: ~130w

Beat -- CTA Bridge

Point: Rules define what Claude can do. Next: Skills -- how to define what Claude knows how to do especially well, so you never have to re-explain it.

Visual: Preview clip from AI Skills Explained

Target words: ~60w

ANALOGY BANK

- Rules file: Like terms of service for an employee -- you do not tell a contractor every possible thing to

do, but you do tell them the non-negotiables: always do X, never do Y, and ask before Z.

- Rules vs permissions: Like a door lock (permissions -- cannot enter at all) vs a house rule (rules -- knock before entering the bedroom). Both control behaviour, at different levels of the system.

THESIS LINE LOCATION

Beat 4 -- The insight is: always, never, escalate covers 90% of what any rules file needs -- and the shorter and more specific the rules, the more reliably Claude follows them.

FRAMEWORK PHRASE

Always, never, escalate

SPECIFICITY BANK

- Rules live in CLAUDE.md -- globally at root, locally in subfolders
- Claude reads the most local CLAUDE.md first, then parent directories up to root
- Rules files persist through context compaction -- unlike conversation-level instructions
- Can reference specific file paths: "never modify /config/production.json"
- Rules are plain English -- no special syntax required

CONSTRAINTS

- Do not re-explain permissions in detail -- reference the previous video
- Do not go into n8n or external automation rule systems -- keep it Claude-native
- Keep examples specific and real -- not abstract ("never delete files") but precise ("never delete files in /Output/")
- Do not explain hooks -- those are the next-next video

CTA DIRECTION

Rules define what Claude can do. Next: Skills -- files that define what Claude knows how to do really well, so you never have to explain the same task twice.

PERSONAL ANGLE [DANIEL TO FILL]

Suggested: A specific rule you added after Claude did something unwanted -- what the rule was and how immediately it changed the behaviour.

VULNERABILITY MOMENT [DANIEL TO FILL]

Suggested: An agent that modified or deleted something it should not have because you had no explicit "never" rule -- and the specific line you added to prevent it.

Note: belongs in Beat 1